

Progress Report: Re-Evaluating LLM Loop Invariant Generation & Repair with CeGIS

Kaustubh Srivastava(B23141) Bharat Agrawal(B23341)
Computer Science and Engineering

1 Overview

This report presents a comparative evaluation based on the benchmark study by Akhond et al. [1]. We re-implemented their top-performing invariant synthesis and repair setups using updated model configurations, introduced our new technique, and benchmarked these approaches across both the original dataset and a newly curated dataset.

1.1 Model Infrastructure & Routing

To efficiently leverage available free API tiers efficiently, our re-implementations and evaluations utilized a round-robin model routing approach, distributing queries across multiple open-source and frontier models including OpenAI’s *GPT-5.6 Luna (High)*, Google’s *Gemini 3.7 Flash (High)* and Anthropic’s *Claude Sonnet 4.6*.

2 Proposed Technique: CeGIS

- **Failure History:** *In RQ2.2, only the counterexample for most recent failure was provided. Consequently, over multiple turns, the LLM would “forget” earlier failures and oscillate back. So, we pass the entire history of previous failures and attempts in each prompt.*
- **Tabu List:** *We maintain a set of previously failed invariants and if the LLM regenerates something from this set, we make the LLM retry some different invariant(with higher temperature) to prevent falling in an infinite loop.*
- **Syntactic Few-Shot Examples:** *As seen from RQ1, providing few-shot examples to the LLM helped improve results. So, for the invariant repair problem as well, we provided the LLM with few-shot examples based on syntactic similarity.*
- **Anti-Overfitting Instructions:** *To prevent the LLM from moving towards reactive point patching and suggesting repairs like $\text{var!} = n$ to patch a specific counterexample, we gave it specific instructions to find true general algebraic boundary conditions.*

3 Dataset Curation

We constructed our benchmark set by combining the LoopInvGen dataset [2] (which itself aggregates SyGuS-COMP, Code2Inv [3], ASE FiB [4], SV-COMP [5], and other sources) with the official SyGuS-COMP benchmark archive [6, 7]. After deduplicating problems common to both sources, this yields a total of **1,020** unique problems, out of which **461** were already verified in prior works. This combined dataset includes problems from the LIA (linear integer arithmetic), ALIA (array-based) and NIA (nonlinear integer arithmetic) theories, making it a diverse and challenging benchmark for evaluating loop invariant synthesis. From this curated collection, we ran our evaluation tests on a subset of **50** problems.

4 Comparative Evaluation

The table below compares the maximum success rates achieved across the original paper baselines [1], our re-implementation on the paper’s benchmark, and our evaluation on the newly curated dataset.

Setup			RQ1.1			RQ1.3			RQ1.5		
			$k = 10$	$k = 30$	$k = 50$	$k = 10$	$k = 30$	$k = 50$	$k = 10$	$k = 30$	$k = 50$
Paper Values (Old Models)			27%	41%	49%	61%	72%	76%	57%	73%	78%
Values (New Models)			80.9%	80.9%	80.9%	87.1%	87.1%	87.1%	100%	100%	100%
Our Benchmark (50 Problems)			74%	74%	74%	80%	80%	80%	80%	80%	80%

Table 1: Comparative performance of loop invariant generation capabilities across setups and datasets.

Setup		RQ2.2		CeGIS	
		$Turns = 1$	$Turns = 3$	$Turns = 1$	$Turns = 3$
Paper Values (Old Models)		16%	—	—	—
Values (New Models)		43.3%	71.4%	99.5%	100%
Our Benchmark (50 Problems)		31%	47%	90%	96%

Table 2: Comparative performance of multi-turn loop invariant repair capabilities across setups and datasets.

4.1 Qualitative Analysis of CeGIS Invariants

Beyond binary verification outcomes, we performed a qualitative classification of the invariants generated by CeGIS relative to the ground-truth specifications. We categorize the synthesized solutions into four structural types: (1) **Exact Match**: identical to the ground truth; (2) **Superset**: contains all ground-truth clauses alongside extra predicates; (3) **Subset**: forms an exact subset, omitting some clauses without introducing extraneous elements; and (4) **Mixed**: contains a combination of missing and extraneous clauses. Table 3 illustrates representative examples for each category.

Problem Name	Classification	Extra Clauses in Generated Result	Missing Clauses in Generated Result
treax1.s1	Exact Match	NONE	NONE
45_conf5.s1	Superset	(= 1 conf_0); (= 1 conf_3); (= 2 conf_4); (< 0 n)	NONE
vardep.s1	Subset	NONE	(or (< x y) (< x z) (= (- y x) 2) (= (- z x) 3) (= 0 x))
95_conf1.s1	Mixed	(= 1 y); (= i j)	(= (* i y) j)

Table 3: Representative examples of structural variations in CeGIS-generated invariants compared to ground truth.

4.2 Execution Time Analysis

Evaluating the complete test set of 50 problems introduces distinct computational overheads across different synthesis and repair strategies. All experiments were executed using a round-robin model routing approach across multiple frontier and open-source models.

For single-shot generation approaches (RQ1), where 50 candidate invariants were sampled per problem to check for at least one valid solution, execution times remained relatively efficient: **17m 58s** for RQ1.1, **13m 43s** for RQ1.3 (few-shot), and **16m 47s** for RQ1.5 (integrated).

In contrast, iterative multi-turn repair pipelines incurred higher latency due to sequential error analysis, prompt generation, and solver verification rounds. Specifically, RQ2.2—which evaluated multi-turn repairs starting from two incorrect invariants per problem across up to three turns—took **1h 9m 58s**. Meanwhile, our proposed CeGIS framework, which begins with a single initial invariant per problem and refines it over up to three turns using feedback history and a tabu list, completed in **44m 25s**. Note that due to the round-robin fashion of utilizing various models, the total time taken is inherently flexible and subject to variation depending on which models the majority of API calls happen to be routed to.

5 Prompt Templates

This section details the exact prompt templates used for each experimental research question (RQ) and our proposed CEGIS framework.

5.1 RQ1.1: Initial Invariant Generation

RQ1.1 — Main Prompt (With Instructions)

You are a helpful AI assistant who can generate invariants from program specifications. You have to find a loop invariant, which can be used to verify several programming properties. SMT is a low-level program specification. For the given problem, find the necessary loop invariant that will be verified using the Hoare triple logics. You have to follow the instructions —

1. Make a note of the conditions of the problem.
2. Analyze the problem and make a note of the conditions.
3. Find the invariant that is true for:
 - 3.1 Before the loop execution (pre-f)
 - 3.2 In every iteration of the loop and (trans-f)
 - 3.3 After the loop termination (post-f).
4. Output the inductive loop invariants in the code block.

Additional Rules:

- Do not use any variables or functions, that are not declared in the problem.
- Do not make any assumptions about functions whose definitions are not given.
- All undefined variables contain garbage values. Do not use variables that have garbage values.
- Variables that are not explicitly initialized, could have garbage values. Do not make any assumptions about such values.
- Do not use non-deterministic function calls.

Now create the invariant for the following program: {problem_smt}
Generate the invariant only, no other text or explanation is desired.

5.2 RQ1.3: Few-Shot Invariant Generation

RQ1.3 — Main Prompt

A loop invariant synthesis problem is given below: {problem_smt}

Synthesize a necessary and sufficient invariant that satisfies all 4 verification constraints. Start the invariant with "inv_fun ({var_names}) Bool (" and end with ")).".

Surround the invariant with <code> and </code> only.

You don't need to explain the invariant, just synthesize it. Follow the example solution formats from the following to generate your output:

Here are some examples: {few_shot_examples}

5.3 RQ1.5: Integrated Invariant Generation

RQ1.5 — Main Prompt

You are a helpful AI assistant who can generate invariants from program specifications. You have to find a loop invariant, which can be used to verify several programming properties. SMT is a low-level program specification. For the given problem, find the necessary loop invariant that will be verified using the Hoare triple logics. You have to follow the instructions —

1. Make a note of the conditions of the problem.
2. Analyze the problem and make a note of the conditions.
3. Find the invariant that is true for:
 - 3.1 Before the loop execution (pre-f)
 - 3.2 In every iteration of the loop and (trans-f)
 - 3.3 After the loop termination (post-f).
4. Output the inductive loop invariants in the code block.

Additional Rules:

- Do not use any variables or functions, that are not declared in the problem.
- Do not make any assumptions about functions whose definitions are not given.
- All undefined variables contain garbage values. Do not use variables that have garbage values.
- Variables that are not explicitly initialized, could have garbage values. Do not make any assumptions about such values.
- Do not use non-deterministic function calls.

Now create the invariant for the following program: {problem_smt}

Generate the invariant only, no other text or explanation is desired. Follow the example solution from the following to generate your output:

Here are some examples: {few_shot_examples}

5.4 RQ2.1: Invariant Repair with Error Cause

RQ2.1 — Initial Prompt

[Uses the same initial prompt structure with instructions as defined in RQ1.1]

RQ2.1 — Repair Prompt

For ({problem_name}) You generated the loop invariant {inv}. But it couldn't verify the problem — Cause of error: {error_cause}, Error details: {error_details}. Now repair the invariant, considering the errors in the previous solution.

Surround the repaired invariant with <code> and </code> only.

5.5 RQ2.2: Invariant Repair with Counterexample

RQ2.2 — Initial Prompt

[Uses the same initial prompt structure with instructions as defined in RQ1.1]

RQ2.2 — Repair Prompt

For ({problem_name}) You generated loop invariant {inv}. Failed to satisfy the “condition”: "{error_details}" for the values {counterexample_values}. Now repair the invariant, considering the errors in the previous solution. Surround the repaired invariant with <code> and </code> only.

5.6 CeGIS: Counterexample-Guided Inductive Synthesis

CeGIS — Main Prompt (First Synthesis Attempt)

[Uses the same integrated few-shot prompt structure as defined in RQ1.5]

CeGIS — Repair Prompt (Enhanced Feedback Multi-Turn)

You are an expert formal verification AI assistant synthesizing an inductive loop invariant.

Target Program Specification: {problem_smt}

Reference Working Example: {reference_example}

Feedback on Previous Synthesis Attempts (Turn {turn}): {attempt_history}

Generalization & Inductive Constraints (CRITICAL):

- **DO NOT OVERFIT:** Do NOT create point-wise disjunctions or hardcoded equality checks for specific counterexample numbers.
- **GENERALIZE:** Deduce the general algebraic inequality or equation (e.g. bounds like ≥ 0 , $\leq n$, or linear relations) that naturally holds for ALL loop iterations while excluding counterexample states.
- **INDUCTIVENESS:** The invariant must hold at loop entry, be preserved by the loop body, and guarantee the postcondition.
- **AVOID PREVIOUS MISTAKES:** Do NOT re-submit any previously attempted invariant.

Output the newly refined inductive invariant starting with `(define-fun InvF ({vars_sig}) Bool` and surrounded by `<code>` and `</code>` only.

References

- [1] M. R. Akhond, S. Chakraborty, and G. Uddin, “LLM for loop invariant generation and fixing: How far are we?” *arXiv preprint arXiv:2511.06552*, 2025.
- [2] S. Padhi, R. Sharma, and T. Millstein, “LoopInvGen: A loop invariant generator based on precondition inference,” *arXiv tech report 1707.02029*, 2017.
- [3] X. Si, H. Dai, M. Raghothaman, M. Naik, and L. Song, “Learning loop invariants for program verification,” in *NeurIPS*, 2018.
- [4] S.-W. Lin, J. Sun, H. Xiao, Y. Liu, D. Sanan, and H. Hansen, “FiB: Squeezing loop invariants by interpolation between forward/backward predicate transformers,” in *Proc. ASE*, pp. 793–803, 2017.
- [5] D. Beyer, “Automatic verification of C and Java programs: SV-COMP 2019,” *TACAS Tech. Rep.*, 2019.
- [6] SyGuS-Org, “SyGuS-COMP benchmarks repository,” 2023. [Online]. Available: <https://github.com/SyGuS-Org/benchmarks>
- [7] R. Alur, R. Bodik, G. Juniwal, M. M. Martin, M. Raghothaman, S. A. Seshia, R. Singh, A. Solar-Lezama, E. Torlak, and A. Udupa, “Syntax-guided synthesis,” in *Proc. FMCAD*, IEEE, 2013.